Reporte: Juego UNO en red (borrador)

> **Borrador.** Lo marcado con [COMPLETAR] lo debe llenar el equipo (datos del curso, diagrama E-R, imagen de ramas, capturas). El resto sale del código y del historial de Git del repositorio `CrAsI-fr/Juego-UNO-P3`.

Materia: [COMPLETAR] · **Profesor:** [COMPLETAR] · **Fecha de entrega:** [COMPLETAR]

Integrantes: Victor [COMPLETAR apellidos] · Joaquin [COMPLETAR] · Ximena [COMPLETAR] · Angel [COMPLETAR]

---

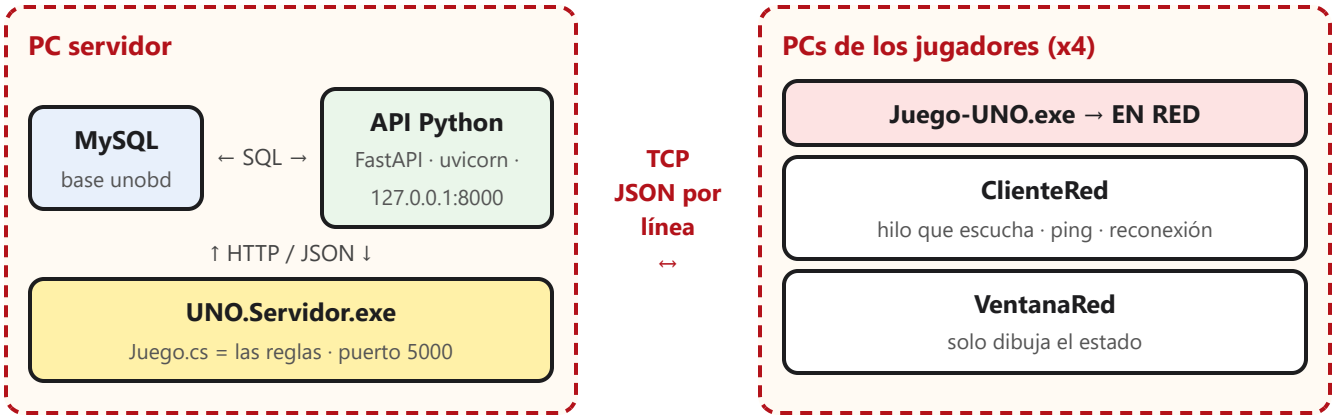
1. Descripción del proyecto

Juego de UNO para 4 jugadores hecho en C# (Windows Forms, .NET Framework 4.7.2), con dos modos:

- **Una sola computadora (JUGAR):** una persona controla a los 4 jugadores y todas las cartas se ven boca arriba.
- **En red (EN RED):** cada jugador usa su propia computadora y solo ve sus cartas. Un programa servidor (`UNO.Servidor`) coordina la partida por sockets TCP.

Todas las partidas y jugadas se guardan en una base de datos MySQL (`unobd`). Por requisito del curso, el código en C# no se conecta directo a MySQL: lo hace a través de una API en Python (FastAPI).

2. Arquitectura



Componente	Tecnología	Responsabilidad
`UNO.Comun`	Biblioteca C#	Reglas del juego (`Juego`), modelos (`Carta`, `Jugador`, `Jugada`), protocolo de mensajes, conexión TCP y cliente de la API
`UNO.Servidor`	Consola C#	Única autoridad de la partida: valida cada jugada, lleva los tiempos y manda a cada jugador lo que puede ver

Componente	Tecnología	Responsabilidad
`Juego-UNO`	Windows Forms	Pantallas: inicio, mesa local, mesa en red, sala de espera, selección de color, ganador y fin
`main.py` + `db.py`	Python, FastAPI, uvicorn	API HTTP/JSON entre C# y MySQL
`scriptuno.sql`	MySQL 8.0	Tablas, restricciones, vistas y las 60 cartas

Decisiones de diseño:

- **Servidor autoritativo:** las reglas viven en una sola clase (`Juego`) que usan tanto el modo local como el servidor. El cliente en red no decide nada; manda intenciones y dibuja el estado que recibe. Así no se puede hacer trampa ni se desincronizan las computadoras.
- **Cada jugador solo recibe su mano:** de los rivales solo viaja cuántas cartas tienen.
- **Sockets TCP con un mensaje JSON por línea:** sencillo de leer y depurar, sin librerías adicionales (`System.Net.Sockets` y Newtonsoft.Json).
- **La API solo la usa el servidor**, en la misma PC: no hace falta abrirla a la red.

3. Funcionamiento en red

El detalle de cada mensaje está en `PROTOCOLO.md`. Resumen:

1. Cada jugador se conecta con su nombre y se sienta en la sala de espera (4 lugares).
2. Con 4 jugadores, el servidor registra la partida en la API, reparte y manda a cada quien su estado.
3. En su turno, el jugador tira, roba o pasa. El servidor valida y, si se puede, avisa a todos y manda el estado nuevo; si no, le explica el motivo solo a ese jugador.
4. Al quedarse con una carta hay 4 segundos para decir UNO; si no, roba 2 cartas de castigo.
5. Al terminar, cada jugador elige jugar otra partida con la misma mesa o salir.

Tolerancia a fallas:

- **Ping cada 5 s:** si un jugador no responde en 15 s, su conexión se da por muerta.
- **Reconexión:** a quien pierde la conexión se le guarda el lugar 60 s. Su juego intenta volver solo con un *token* y recupera su mano.
- **Inactividad:** si quien tiene el turno no juega en 70 s, recibe una alerta y tiene 20 s para confirmar que sigue; si no, se le expulsa.
- **Registro asíncrono:** las jugadas se guardan en la API desde otro hilo, en orden, para que una API lenta no congele la partida.

4. Base de datos

[COMPLETAR: insertar el diagrama E-R]

Tabla / vista	Contenido
`carta`	Las 60 cartas: tipo (ordinaria/especial), color, efecto y número, con una restricción de consistencia

Tabla / vista	Contenido
`jugador`	Nombre único de cada jugador
`partida`	Fecha de inicio y fin, y ganador (vacío si se abandonó)
`partida_jugador`	Qué jugadores participaron y en qué posición (define el orden de los turnos)
`jugada`	Log de cada movimiento: reparto, carta inicial, tirar, robar, pasar, decir UNO y castigos
`vista_historial`, `vista_log`	Partidas jugadas, ganadas y perdidas por jugador; log legible de una partida

5. Reglas implementadas y decisiones del equipo

Reglas: validación por color, número, símbolo o comodín; reversa, bloqueo, +2 y +4 (el siguiente roba y pierde turno); comodines con elección de color; robo y pasar; decir UNO con castigo de 2 cartas.

Diferencias con el UNO oficial (decisiones del equipo):

- Mazo de **60 cartas** en lugar de 108 (sin ceros y con menos especiales).
- La carta inicial siempre es de número.
- El +4 se puede tirar aunque se tenga una carta del color en juego.
- Los +2 no se acumulan.
- Se puede robar sin límite en el turno; se puede pasar después de robar.
- El castigo por no decir UNO es automático (no hay que acusar).

6. División de tareas

Según el historial de Git del repositorio (`git shortlog`, PRs #1 a #5). [COMPLETAR: revisar y ajustar con lo que cada quien hizo fuera de Git, como la base de datos, la API y la versión de una PC.]

Integrante	Trabajo en la versión en red
Victor	Separación de las reglas en la clase `Juego`; servidor y cliente TCP; protocolo; integración y revisión de PRs; correcciones de los PRs #2, #4 y #5; reconexión, ping, registro asíncrono, otra partida y mesa con rivales a los lados
Joaquin	Proyecto inicial en el repositorio; `.env.ejemplo`; desconexión por inactividad y su pantalla de alerta (PR #3)
Ximena	Cartas de los rivales boca abajo en la mesa en red (PR #5); selección de color (`FormElegirColor`, versión de una PC)
Angel	Mazo como imagen, botones con estilo, cuenta regresiva del UNO, sala de espera visual, reintento de nombre y pantalla de fin (PR #1)

7. Historial de ramas y commits

[COMPLETAR: insertar la imagen de ramas (por ejemplo, la gráfica de GitHub o `git log --graph`) y la lista de commits desde el 21/09/2026 del repositorio anterior.]

PR	Rama	Autor	Contenido
#1	`ui-angel`	Angel	Sala de espera, mazo como imagen, cuenta del UNO, pantalla de fin
#2	`correcciones-red`	Victor	Nombres y motivo de rechazo como datos del protocolo; ajustes de la mesa
#3	`rama_joaquin`	Joaquin	Desconexión por inactividad
#4	`correcciones-inactividad`	Victor	Aviso a los demás jugadores, documentación del protocolo, limpieza
#5	`xop`	Ximena, Victor	Rivales boca abajo; correcciones de compilación y de la mesa
—	`victor_final`	Victor	Reconexión, ping, registro asíncrono, otra partida, rivales a los lados, avisos visibles

8. Problemas encontrados y cómo se resolvieron

Problema	Causa	Solución
Visual Studio marcaba error en Newtonsoft.Json	La carpeta `packages/` no estaba en el repositorio	Restaurar paquetes NuGet al compilar; se quitó MySqlConnector, que ya no se usaba, y se agregó `.gitignore`
No se podía reutilizar la lógica en el servidor	Las reglas estaban mezcladas con la ventana (`Form1.cs`)	Se separaron en la clase `Juego`, sin interfaz ni base de datos
No se reconocía `uvicorn`	El entorno virtual se creó, pero `pip install` se corrió fuera de él	Instalar con `.venv\Scripts\python.exe -m pip install -r librerias.txt`
El servidor no existía en `bin\Debug`	Se intentó ejecutar sin compilar la solución completa	Compilar la solución (3 proyectos) antes de ejecutar
Dudas sobre qué IP compartir	La PC mostraba la IP de un adaptador virtual (`192.168.56.x`)	Usar la IP del adaptador Wi-Fi/Ethernet de la red común
Un nombre con coma descuadraba la sala	El cliente sacaba los nombres de un texto pensado para leerse	El servidor manda la lista de nombres como dato
El reintento de nombre dependía de la redacción del error	El cliente buscaba palabras en el texto	Código de error `nombre_invalido` en el protocolo

Problema	Causa	Solución
La mesa se quedaba esperando sin aviso	La alerta de inactividad solo le llegaba al jugador inactivo	Aviso a los demás jugadores
El proyecto no compilaba tras un merge	Se borró por accidente la declaración de `FormInicio`	Restaurarla; revisar que compile antes de cada PR
La mesa en red tronaba al abrirse	Variables locales con el mismo nombre que los campos de la ventana	Asignar a los campos
El juego acumulaba controles en memoria	`Controls.Clear()` no libera los controles	Liberarlos con `Dispose()` en cada redibujo
Rivales en desorden y cartas que no cabían	Acomodo con `Dock` y tamaño fijo	Orden de turno y cartas encimadas según el espacio
Si la API tardaba, se congelaba la mesa	Cada jugada esperaba a MySQL dentro del candado del servidor	Cola de registro en otro hilo
Si se caía la red, la partida terminaba para todos	No había forma de recuperar el lugar	Token y reconexión automática, con 60 s de espera
Una conexión muerta tardaba en detectarse	TCP no avisa si se desconecta el cable	Ping cada 5 s y límite de 15 s sin respuesta
[COMPLETAR]	Problemas de la prueba en el laboratorio (red, firewall, etc.)	

9. Pruebas realizadas

- Simulación de 2000 partidas sobre la clase `Juego`, sin perder cartas.
- Partidas completas en una PC con la mesa en red real contra el servidor y jugadores automáticos.
- Nombre repetido, entrar con la partida empezada, desconexión, servidor caído, jugador que no responde al ping, reconexión, inactividad, otra partida y una API que tarda 1,5 s por jugada.
- Prueba del equipo entre varias PCs con la API real. **[COMPLETAR: fecha, número de PCs y resultado; repetir con la versión final.]**

10. Limitaciones y trabajo futuro

- Al empezar cada partida, el servidor sí espera a la API (registro de jugadores y reparto).
- Si la API está muy lenta, el log se atrasa; si el servidor se cierra antes de vaciar la cola, se pierden esas jugadas del log.
- Una sola mesa por servidor; si el servidor se reinicia, la partida en curso se pierde.
- El modo local y el modo en red dibujan la mesa por separado.

11. Cómo ejecutarlo

Resumen; el detalle está en `RED.md`.

1. En la PC servidor: MySQL con `scriptuno.sql`, `.env`, la API (`.venv\Scripts\python.exe -m uvicorn main:app`), el puerto 5000 abierto en el firewall y `UNO.Servidor.exe`.
2. En cada PC: compilar la solución, abrir `Juego-UNO.exe` → **EN RED**, escribir la IP del servidor y un nombre.

Anexo: capturas

[COMPLETAR: pantalla de inicio, sala de espera, mesa en red, alerta de inactividad, ganador y otra partida.]